

EXPERIMENT 11

Student Name: Ridhima Gulati

Branch: AIT-CSE (Cyber Security)

Semester: 5

Subject Name: ADBMS

UID: 24BCY70132

Section/Group: 24AIT_KRG-G2

Date: 14-09-2026

Subject Code: 24CSP-333

EXPERIMENT: Advanced SQL Queries

AIM: To study and implement SQL Window Functions such as ROW_NUMBER(), RANK(), DENSE_RANK(), LAG(), and LEAD() to perform analytical operations, salary comparisons, and department-wise ranking on employee data.

Question 1: A company stores the monthly salary history of its employees in an EmployeeSalaryHistory table. Write an SQL query to find employees whose salary has increased in every recorded month compared to the immediately previous month.

For each salary record, compare the current month's salary with the employee's previous month's salary using the LAG() window function. Display only those employees for whom all later salary records are greater than their respective previous salary.

Output should display:

- Employee ID
- Employee Name

Solution: CREATE TABLE EmployeeSalaryHistory (

record_id INT PRIMARY KEY,

employee_id INT NOT NULL,

employee_name VARCHAR(50) NOT NULL,

salary_month DATE NOT NULL,

EXPERIMENT 11

```
salary INT NOT NULL

);

INSERT INTO EmployeeSalaryHistory
(record_id, employee_id, employee_name, salary_month, salary)
VALUES
(1, 101, 'Alok', '2026-01-01', 50000),
(2, 101, 'Alok', '2026-02-01', 52000),
(3, 101, 'Alok', '2026-03-01', 55000),
(4, 102, 'Priya', '2026-01-01', 60000),
(5, 102, 'Priya', '2026-02-01', 58000),
(6, 102, 'Priya', '2026-03-01', 62000),
(7, 103, 'Raj', '2026-01-01', 45000),
(8, 103, 'Raj', '2026-02-01', 47000),
(9, 103, 'Raj', '2026-03-01', 49000),
(10, 104, 'Sneha', '2026-01-01', 70000),
(11, 104, 'Sneha', '2026-02-01', 70000),
(12, 104, 'Sneha', '2026-03-01', 72000);

select * from EmployeeSalaryHistory;

select employee_id, employee_name
from(
select *, LAG(salary) over(partition by employee_id
ORDER BY salary_month
```

EXPERIMENT 11

```

)prev_salary
from EmployeeSalaryHistory
)E
group by employee_id,employee_name
having count(*) filter(
WHERE salary <= prev_salary
)=0;

```

OUTPUT:

Data Output	Messages	Notifications
≡+ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈 SQL		
	employee_id integer	employee_name character varying (50)
1	103	Raj
2	101	Alok

Question 2: A company maintains employee details in an Employee table and department information in a Department table.

Write an SQL query to find employees whose salaries belong to the top three distinct salary levels within their respective departments.

If multiple employees have the same salary, they should receive the same salary rank and all of them should be included in the result.

Output should display:

- Department Name
- Employee Name
- Salary

Solution: CREATE TABLE Department (

EXPERIMENT 11

```
id INT PRIMARY KEY,  
name VARCHAR(50) NOT NULL  
);  
INSERT INTO Department (id, name)  
VALUES  
(1, 'IT'),  
(2, 'Sales'),  
(3, 'HR');
```

```
CREATE TABLE Employee (  
id INT PRIMARY KEY,  
name VARCHAR(50) NOT NULL,  
salary INT NOT NULL,  
departmentId INT,  
FOREIGN KEY (departmentId)  
REFERENCES Department(id)  
);
```

```
INSERT INTO Employee  
(id, name, salary, departmentId)  
VALUES  
(1, 'Max', 90000, 1),  
(2, 'Jim', 85000, 1),  
(3, 'John', 85000, 1),
```

EXPERIMENT 11

(4, 'Alex', 75000, 1),
(5, 'Sam', 70000, 1),
(6, 'Henry', 80000, 2),
(7, 'Susan', 75000, 2),
(8, 'David', 70000, 2),
(9, 'Robert', 65000, 2),
(10, 'Tina', 60000, 3),
(11, 'Rita', 55000, 3),
(12, 'Manoj', 50000, 3);

```
SELECT D.name AS Department,  
       E.name AS Employee,  
       E.salary  
FROM (  
    SELECT *,  
           DENSE_RANK() OVER (  
               PARTITION BY departmentId  
               ORDER BY salary DESC  
           ) AS rnk  
    FROM Employee  
) E  
JOIN Department D  
ON E.departmentId = D.id  
WHERE rnk <= 3;
```

EXPERIMENT 11

OUTPUT:

Data Output Messages Notifications			
	department character varying (50) 🔒	employee character varying (50) 🔒	salary integer 🔒
1	IT	Max	90000
2	IT	Jim	85000
3	IT	John	85000
4	IT	Alex	75000
5	Sales	Henry	80000
6	Sales	Susan	75000
7	Sales	David	70000
8	HR	Tina	60000
9	HR	Rita	55000
10	HR	Manoj	50000

LEARNING OUTCOMES:

1. Understand the concept and purpose of SQL Window Functions for performing analytical operations.
2. Apply window functions such as ROW_NUMBER(), RANK(), and DENSE_RANK() to rank records within groups.
3. Use LAG() and LEAD() to compare current records with previous or subsequent records.
4. Analyze employee salary data using PARTITION BY and ORDER BY for department-wise comparisons.
5. Develop efficient SQL queries to solve real-world analytical problems involving salary trends, rankings, and sequential data.